

DSS-Becas — Guía de estudio

Sistema de Soporte a Decisiones para asignación y seguimiento de becas · Resumen para estudiar rápido (5 integrantes)

1. ¿Qué es y qué hace?

Sistema que **recomienda** a quién dar una beca y **vigila** que siga cumpliendo. La decisión final la toma una persona.

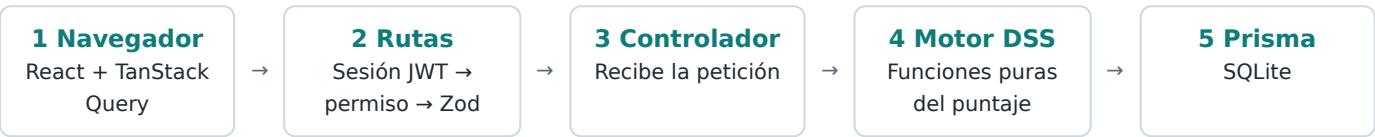
Etapa 1: ASIGNAR		Etapa 2: SEGUIR
Quién	Postulantes sin beca	Becarios con asignación aprobada
Pantallas	Estudiantes → Evaluación DSS → Becas	Seguimiento
Pregunta	¿Quién merece la beca?	¿Sigue cumpliendo?
Regla	Puntaje ponderado + elegibilidad	Promedio del periodo ≥ 51

Regla para recordar: Evaluación DSS decide si le dan la beca; Registrar periodo comprueba si sigue mereciéndola. Solo **Becas** puede quitarla.

2. Tecnologías

Capa	Tecnología	Para qué
Frontend	React 18 + TypeScript + Vite 5	Interfaz por componentes, tipada
Rutas	React Router 6	Navegación y rutas protegidas
Datos del servidor	TanStack Query 5	Caché, estados cargando/error, refresco tras guardar
Backend	Node + Express 4 + TypeScript	API REST en JSON
Base de datos	SQLite + Prisma 5	ORM tipado (sin instalar servidor; portable a MySQL)
Validación	Zod	Valida el cuerpo y da errores por campo
Seguridad	JWT + bcryptjs	Sesión por token; contraseñas con hash
Archivos / pruebas	multer / Vitest + supertest	Subida de documentos / pruebas automáticas

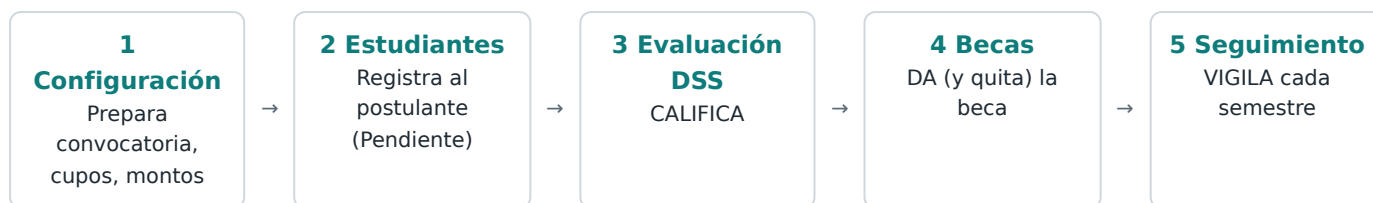
3. Arquitectura y recorrido de una petición



Regla de arquitectura: el cálculo del puntaje vive **solo** en `backend/src/dss/` ; los controladores lo llaman.

Código	Significado
400	Datos inválidos (Zod), con detalle por campo
401	Sin sesión válida
403	Hay sesión pero el rol no alcanza
404 / 409	No existe / valor repetido o con dependencias (ej. CI duplicado)
500 / 503	Error interno con código ERR-XXXX / base ocupada

4. Recorrido por los 8 módulos



Dashboard e Reportes: solo ver resultados. **Administración:** usuarios y roles.

¿En qué punto se quita la beca?

- **Seguimiento solo avisa:** promedio < 51 = «En riesgo»; dos periodos seguidos = sugerencia de suspender.
- **Becas la quita:** *Rechazar* (evaluador, con justificación ≥ 10 caracteres) o *Revocar* (solo administrador).

5. Motor DSS: cómo se calcula el puntaje

Criterio	Peso	Qué se califica (0-100, más = más a favor)	Tipo
Rendimiento académico	30 %	Qué tan bien le va	beneficio
Asistencia	15 %	Constancia	beneficio
Situación socioeconómica	25 %	Necesidad: ingreso bajo = nota alta	costo
Carga familiar	15 %	Dependientes del hogar	beneficio
Condición vulnerable	15 %	Vulnerabilidad acreditada	beneficio

Puntaje final	Etiqueta
≥ 80	Recomendado
60 - 79.99	En revisión
< 60	En riesgo

Fórmulas: académico = (rendimiento + asistencia)/2 · social = (situación + carga + vulnerable)/3 · final = (rend·30 + asist·15 + sit·25 + carga·15 + vuln·15)/100.

Ejemplo: 90/80/70/60/50 → (2700+1200+1750+900+750)/100 = **73.0** → **En revisión**. Otro: 100/100/100/0/0 → 70.0.

Tipo beneficio / costo: beneficio = más valor, mejor; costo = más valor, peor (el ingreso). Hoy es informativo: el motor no lo lee.

motor.ts

backend > src > dss > motor.ts

```
18 const redondear2 = (n: number): number => Math.round(n * 100) / 100;
19
20 /** Suma ponderada con los pesos de CRITERIOS. No guarda nada. */
21 export function calcular(criterios: PuntajesCriterio): ResultadoCalculo {
22   validarPesos(CRITERIOS);
23   const peso: Record<keyof PuntajesCriterio, number> = {
24     rendimiento: 30,
25     asistencia: 15,
26     situacion: 25,
27     carga: 15,
28     vulnerable: 15,
29   };
30   const academico = redondear2((criterios.rendimiento + criterios.asistencia) / 2);
31   const social = redondear2(
32     (criterios.situacion + criterios.carga + criterios.vulnerable) / 3,
33   );
34   const final = redondear2(
35     (criterios.rendimiento * peso.rendimiento +
36       criterios.asistencia * peso.asistencia +
37       criterios.situacion * peso.situacion +
38       criterios.carga * peso.carga +
39       criterios.vulnerable * peso.vulnerable) /
40     100,
41   );
42   return {
43     puntaje_academico: academico,
44     puntaje_social: social,
45     puntaje_final: final,
46     recomendacion: recomendar(final),
```

1

2

3

4

6. Reglas del sistema

Regla	Cómo funciona
Elegibilidad	Promedio mínimo 51. Beca Social: ingreso familiar máx. Bs 4000. Si falla → «No elegible» con motivo.
Ranking	Una fila por estudiante (mejor puntaje). Desempate: promedio ↓, luego menor ingreso.
Cupos	«Generar asignación» toma solo los cupos libres del tipo de beca (Excelencia Bs 600, Social Bs 400).
Presupuesto	Usado = aprobadas × monto. Disponible = presupuesto de la convocatoria – usado.
Decisión	Rechazar u observar exige justificación de 10+ caracteres.
Seguimiento	Estado calculado por el servidor: el usuario solo escribe el promedio.
Evaluar pendientes	Deriva criterios: rendimiento = promedio, asistencia 90, situación inversa al ingreso, carga 70, vulnerable 60.

7. Pantallas: qué señalar

DSS-Becas · Bienestar Universitario

Dashboard DSS
Estudiantes
Evaluación DSS
Becas
Seguimiento
Reportes
Administración
Configuración

DSS-Becas · Sistema de Soporte a Decisiones

Evaluación DSS - Nueva evaluación

Criterios ponderados

Estudiante	Maria Fernández	Fecha	09/29/2026
Rendimiento académico	90	30%	
Asistencia	80	15%	
Situación socioeconómica	70	25%	
Carga familiar	60	15%	
Condición vulnerable	50	15%	

Calcular puntaje DSS

Resultado de la evaluación

Puntaje DSS
73.0

Recomendación: **En revisión**

Comparación con umbral

- Recomendado (>= 80)
- En revisión (60-79)
- En riesgo (< 60)

Evaluación DSS

- 5 notas 0-100, pesos fijos
- Resultado 73.0 → En revisión
- Promedio < 51 → «No elegible»

DSS-Becas · Bienestar Universitario

Dashboard DSS
Estudiantes
Evaluación DSS
Becas
Seguimiento
Reportes
Administración
Configuración

DSS-Becas · Sistema de Soporte a Decisiones

Gestión de becas

Convocatoria Becas 2025 | Excelencia | **Generar asignación** | **Evaluar pendientes** | Exportar CSV | Exportar PDF

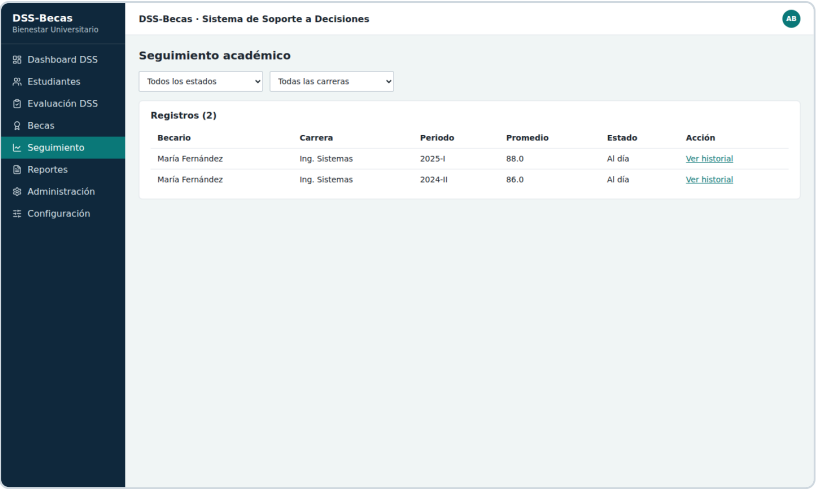
Cupos ocupados: **1** | Cupos disponibles: **19** | Presupuesto usado (Bs): **600** | Presupuesto disponible (Bs): **49400**

Ranking de candidatos (27)

#	Estudiante	Carrera	Puntaje	Recomendación
1	Maria Fernández	Ing. Sistemas	92.5	Recomendado
2	Marco Apaza	Derecho	91.0	Recomendado
3	Andrés Ríos	Economía	90.4	Recomendado
4	Fernanda Calle	Medicina	89.9	Recomendado
5	Jorge Quispe	Medicina	89.1	Recomendado
6	Diego Condori	Ing. Sistemas	88.3	Recomendado
7	Paola Vargas	Enfermería	87.2	Recomendado
8	Natalia Ortiz	Contaduría	86.1	Recomendado
9	Pedro Choque	Ing. Civil	85.6	Recomendado
10	Carmen Poma	Ing. Civil	84.5	Recomendado
11	Martín Paredes	Ing. Civil	83.7	Recomendado
12	Daniela Miranda	Ing. Sistemas	82.8	Recomendado
13	Ana Rojas	Derecho	81.4	Recomendado
14	Beatriz Mamani	Psicología	80.5	Recomendado

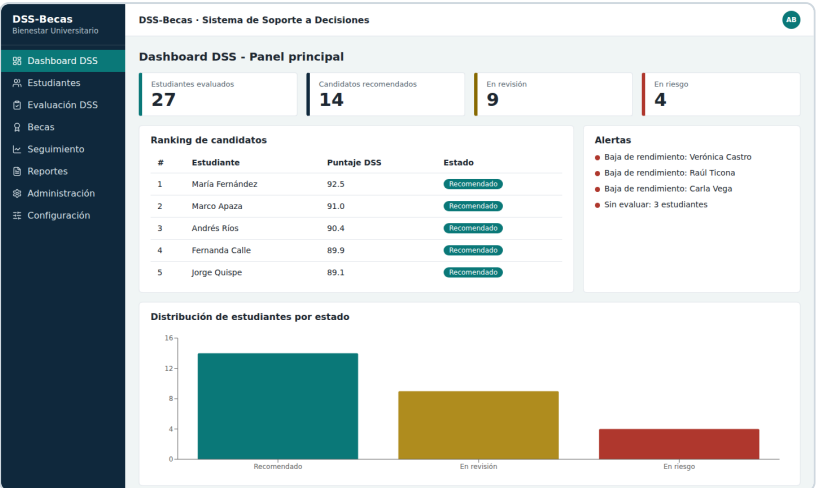
Becas

- Convocatoria + tipo de beca
- Cupos y presupuesto usado/disponible
- Generar → top-N por cupos
- Aprobar · Rechazar · Observar · Revocar



Seguimiento

- Un periodo por fila con su promedio
- Estado automático (< 51 = En riesgo)
- Ver historial: línea de tiempo y sugerencia de suspensión



Dashboard

- KPI de evaluados y estados
- Top 5: mejor puntaje por estudiante
- Alertas y distribución

8. Seguridad y roles

Módulo	Administrador	Evaluador	Consulta
Estudiantes	Crear, editar, eliminar	Crear, editar	Solo ver
Evaluación DSS	Todo	Evaluar	Oculto
Becas	Generar, decidir, revocar	Aprobar / rechazar / observar	Solo ver
Seguimiento	Todo	Registrar periodos	Solo ver
Reportes	Ver, CSV	Ver, CSV	Ver, CSV
Configuración / Administración	Todo	Oculto	Oculto

- **JWT** de 8 h con `token_version`: al cambiar contraseña o usuario, las sesiones viejas dejan de servir.
- **requireAuth** recarga al usuario de la BD en cada petición (rol y estado aplican al instante).
- **bcrypt**: contraseñas con hash y sal; nunca se devuelven al cliente.
- **Límite de intentos**: 5 fallos en 10 min → bloqueo (en memoria).
- **Permisos** en `backend/src/auth/permisos.ts` (única fuente). Un test verifica que ninguna ruta quede sin permiso.
- El frontend solo oculta botones; la seguridad real es el 403 del servidor.

9. Base de datos

Tabla	Para qué sirve
Estudiante	Datos personales, académicos y socioeconómicos (CI y correo únicos)
Evaluacion / Resultado	Puntajes por fecha / etiqueta vigente (una por estudiante → <code>upsert</code>)
Asignacion	Estudiante ↔ convocatoria ↔ tipo de beca (combinación única)
Seguimiento	Promedio por periodo de cada asignación
Carrera, TipoBeca, Convocatoria, Criterio	Catálogos configurables
Evento, Documento, Usuario	Historial, archivos adjuntos y acceso

- **Cascade**: al eliminar un estudiante se borran sus evaluaciones, becas, documentos, eventos y asignaciones.
- **Restrict**: no se borra un tipo de beca o convocatoria con asignaciones (409).
- SQLite y no MySQL: no había servidor; basta cambiar `provider` y `DATABASE_URL` (D26).

10. Frontend en corto

Pieza	Qué hace
<code>main.tsx</code>	Anida providers: Query → Toast → Auth → Router
<code>routing/</code>	Rutas anidadas; <code>RequireAuth</code> y <code>RequirePermiso</code> (si no hay permiso: «Acceso denegado»)
<code>services/api/</code>	Un archivo por recurso; <code>client.ts</code> pone el token y convierte errores en <code>ApiError</code> . Modo mock solo con <code>VITE_USE MOCKS=true</code>
<code>state/</code>	<code>AuthContext</code> (sesión), Permisos (<code>usePermiso</code>), Toasts, <code>useDebounce</code>
TanStack Query	<code>queryKey</code> = etiqueta de caché; <code>invalidateQueries</code> tras guardar refresca las otras pantallas
Formularios	Validan en cliente y muestran los errores del servidor por campo; aceptan coma decimal (<code>1.499,99</code>)

11. Pruebas y calidad

Dato	Valor
Pruebas backend (Vitest + supertest)	52 — pasan
Pruebas frontend	11 — pasan
Issues de QA cerrados	#1-#9
Decisiones documentadas	D1-D67 en <code>docs/DECISIONES.md</code>

También: checklist ISO 25010, matriz de trazabilidad, contrato OpenAPI, casos CP-001...CP-004 y metodología *docs-first*.

12. Limitaciones conocidas (conózcanlas antes que el docente)

Limitación	Cómo decirlo / mejora
Los pesos editables en Configuración no cambian el cálculo (el motor usa pesos fijos)	«Los pesos del prototipo son la regla vigente; conectar el motor a la BD es el siguiente paso.»
Un becario aprobado no aparece en Seguimiento hasta registrar su primer periodo	Mostrar a todos los becarios aprobados, una fila por becario.
«Generar asignación» controla cupos pero no presupuesto	Validar también el dinero disponible.
El servidor confía en el puntaje que manda el cliente al guardar	Recalcular en el servidor desde los 5 criterios.
«Suspendida» solo se sugiere; la beca se quita en Becas	Botón «Suspend» conectado a Seguimiento.
El «usado» del presupuesto cuenta solo el tipo de beca elegido	Sumar las aprobadas de todos los tipos de la convocatoria.
El estado «Activa» de Estudiantes viene de la tabla Beca, no de las asignaciones	Derivarlo de asignaciones aprobadas.
Límite de intentos de login en memoria	Se pierde al reiniciar (aceptado en esta fase, D58).

13. Preguntas probables y respuesta corta

Pregunta	Respuesta
¿Dónde está la lógica DSS y por qué ahí?	En <code>backend/src/dss/</code> , funciones puras: aisladas de controladores y SQL, fáciles de probar.
¿Cómo desempata el ranking?	Puntaje ↓, promedio ↓, menor ingreso familiar.
¿Qué es «No elegible»?	Promedio < 51 o (beca Social) ingreso > Bs 4000, aunque el puntaje sea alto.
¿Cómo protegen la API?	Login → JWT; requireAuth; requirePermiso por ruta; test de cobertura de rutas.
¿Por qué <code>token_version</code> ?	Para invalidar sesiones sin lista negra.
¿Por qué validan en cliente y servidor?	Cliente = comodidad; servidor = seguridad (no se confía en el cliente).
¿Qué bug grave resolvieron?	Error 500 al guardar: coma decimal, cadenas vacías en campos únicos, carpeta <code>uploads</code> inexistente y SQLite bloqueado (D51-D56).
¿Qué incidente tuvieron con las pruebas?	Un import estático escribió en la base real; se corrigió con import dinámico (D62/D63).
¿Qué es el modo mock?	Demo sin backend; nunca se activa solo ni hay caída silenciosa.
¿Cómo evitan pasarse de cupos?	Solo toman <code>cupos - ocupados</code> candidatos y hay restricción única en la BD.
¿Quién decide el estado del seguimiento?	El servidor, por regla; el usuario solo digita el promedio.

14. Reparto de estudio (5 integrantes)

#	Bloque	Archivos clave
1	Motor DSS y pruebas	<code>backend/src/dss/*</code> , <code>tests/api.test.ts</code> , <code>tests/qa/</code>
2	Base de datos y API de negocio	<code>schema.prisma</code> , <code>seed.ts</code> , controladores de estudiantes, evaluaciones, asignaciones, seguimiento
3	Seguridad, validación y errores	<code>auth/*</code> , <code>middlewares/*</code> , <code>validators/schemas.ts</code> , tests de auth y permisos
4	Frontend: estructura y servicios	<code>main.tsx</code> , <code>routing/</code> , <code>state/</code> , <code>services/api/</code>
5	Frontend: pantallas y documentación	<code>pages/</code> , <code>components/</code> , <code>docs/</code> , SRS, OpenAPI

15. Autoexamen (respondan sin mirar)

1. Calculen 100/100/100/0/0 → académico, social, final y etiqueta. (100 / 33.33 / 70.0 → En revisión)
2. ¿En qué orden pasan los middlewares de `POST /api/evaluaciones`? (`auth` → `permiso` → `Zod` → `controlador`)
3. ¿Qué se borra al eliminar un estudiante? ¿Y qué impide borrar?
4. ¿Qué pasa si el admin cambia la contraseña de alguien con sesión abierta?
5. ¿Diferencia entre 401 y 403?
6. ¿Qué hace `invalidateQueries(["dashboard"])`?

7. ¿Cómo se sabe cuánto presupuesto queda? ¿Cuándo baja?
8. ¿Dónde se quita una beca y quién puede?
9. Nombren 3 limitaciones y su mejora.

16. Cómo ejecutarlo y demo

`npm run setup` (instala, migra, siembra) · `npm run dev` (backend :3001 + frontend :5173) · `npm test` en cada carpeta. Usuarios de demo: **admin** / Admin2026! · **evaluador** / Evaluador2026! · **consulta** / Consulta2026!.

Demo: Login admin → Nuevo estudiante → Evaluación (90/80/70/60/50 = 73.0) → Becas: generar y rechazar sin/con motivo → Seguimiento: promedio 45 = En riesgo → Reportes CSV → entrar como consulta y ver que no hay botones.